Chasing the Moment of Impact

Collision Detection and Iterative Contact Resolution on CPU and CUDA

Justin-Marian Popescu

Proiect GPGPU

Rezumat

Când o bilă lovește o cutie, simulatorul trebuie să găsească locul contactului și să calculeze cum se schimbă viteza și rotația fiecărui object. Raportul urmărește acest proces de la aplicarea gravitației până la urma colorată din imagine. Exemplele numerice și fragmentele din cod leagă formulele de implementarea C++ și de shaderele GLSL. Aceleași patru scene au fost rulate pe CPU și CUDA. Toate cele 4 560 de perechi de pași comparate au avut hash-uri de stare identice. În scena cu 4 000 de objects, raportul dintre timpul median pe CPU și cel pe CUDA a fost 1,59.

Cuvinte-cheie rigid-body dynamics, collision detection, impulse, friction, CUDA, impact heatmap

Cuprins

1 De la cădere la impact **1**

2 Rigid-body state and dynamics **1**
2.1 Position, orientation, mass, and inertia . . . 1
2.2 Gravity, damping, and time integration . . . 2
2.3 Quaternion rotation . . . 3
2.4 Energy and diagnostics . . . 3

3 Collision detection **4**
3.1 Broad phase and spatial grid . . . 4
3.2 Narrow phase and contact manifolds . . . 4
3.3 Intersection tests . . . 6
3.4 Continuous collision detection and wall contacts . . . 6

4 Iterative contact solver **7**
4.1 Relative velocity and effective mass . . . 7
4.2 Restitution and normal impulse . . . 8
4.3 Angular impulse and friction . . . 9
4.4 Position correction . . . 10

5 CUDA execution and reproducibility **10**

6 Scene rendering and impact visualization **11**
6.1 From physics state to rendered frame . . . 11
6.2 Soft contact shadows . . . 11
6.3 Impact energy . . . 12
6.4 Collision heatmap . . . 13

7 Interfața viewer-ului **14**

8 Build și rulare **16**
8.1 Build pe Windows . . . 17
8.2 Build pe Linux . . . 17

9 Validare experimentală **17**
9.1 Metodă . . . 17
9.2 Rezultate . . . 18
9.3 Reproducerea comparației . . . 18

10 Concluzii **19**

Sursele proiectului **19**

1 De la cădere la impact

Poziția arată unde se află centrul bilei. Viteza liniară arată în ce direcție și cât de repede se deplasează, iar viteza unghiulară descrie rotația în jurul centrului. Gravitația schimbă viteza pe durata căderii. La lovire, simulatorul aplică un *impulse*, adică o schimbare a cantității de mișcare. Un impuls a cărui linie de acțiune trece pe lângă centru poate pune object-ul în rotație. Termenul *rigid body* desemnează un object căruia modelul îi păstrează forma în timpul acestor mișcări.

Proiectul `chasing-moment-of-impact` folosește un *physics engine* cu pas fix h. La fiecare pas aplică forțele, caută perechi apropiate, verifică formele, rezolvă contactele și actualizează pozițiile și orientările. Viewer-ul desenează starea rezultată. Figura 1 arată incinta și object-urile din scenariul demonstrativ. Engine-ul folosește un nucleu comun C++, un *CPU backend* serial și un *CUDA backend* paralel.

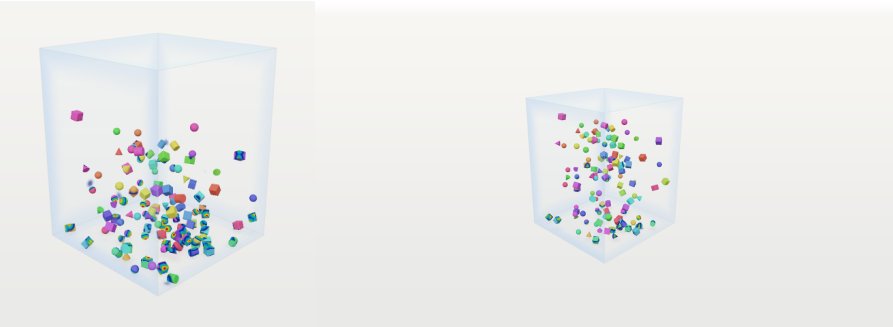


Figura 1: Objects de forme diferite în incintă. În stânga se vede scenariul demonstrativ de aproape, iar în dreapta o rulare CUDA văzută de la distanță.
Imagine din proiect `media/cuda-demo.png` (stânga) și `reports/run-cuda.png` (dreapta).

2 Rigid-body state and dynamics

2.1 Position, orientation, mass, and inertia

Un object este descris prin poziția centrului de masă $\boldsymbol{x}$, orientarea q, *linear velocity* $\boldsymbol{v}$, *angular velocity* $\boldsymbol{\omega}$, masa m și geometria de coliziune. Pozițiile sunt în metri, vitezele liniare în m/s, vitezele unghiulare în rad/s și masele în kilograme. *Linear momentum* se calculează ca $\boldsymbol{P} = m\boldsymbol{v}$, iar *angular momentum* față de centrul de masă este $\boldsymbol{L} = I_w\boldsymbol{\omega}$. Aici I_w reprezintă *inertia tensor* exprimat în *world space*. Masa descrie rezistența la schimbarea translației. Inerția descrie rezistența la schimbarea rotației și depinde de axă și de distribuția masei.

Pentru o sferă omogenă de rază r, valorile *principal moments of inertia* sunt toate $\frac{2}{5}mr^2$. Pentru o cutie de semidimensiuni (b_x, b_y, b_z), tensorul în axele proprii este diagonal, cu

$$I_x = \tfrac{m}{3}(b_y^2 + b_z^2), \quad I_y = \tfrac{m}{3}(b_x^2 + b_z^2), \quad I_z = \tfrac{m}{3}(b_x^2 + b_y^2). \tag{1}$$

De exemplu, $I_x = \int (y^2 + z^2)\,\mathrm{d}m$, iar pentru distribuția uniformă pe intervalul $[-b_y, b_y]$ media lui y^2 este $b_y^2/3$. Engine-ul folosește un *box collider* și pentru cilindri și conuri. Dacă Q transformă axele locale în *world space*, $I_w^{-1} = QI_{\text{local}}^{-1}Q^{\mathsf{T}}$. Valorile stocate sunt m^{-1} și diagonala lui I_{local}^{-1}, folosite direct la aplicarea impulsurilor. Inițializarea acestor valori se găsește în `State::spawn` (**C1**).

`State` deține un *structure of arrays* (SoA), cu câte un tablou pentru fiecare proprietate. `StateView` oferă acces la aceleași date prin *pointers* pe CPU sau CUDA. Contactele sunt înregistrări separate, cu doi indici, un punct, o normală, adâncimea cu semn și impulsurile acumulate. Al doilea indice este negativ când contactul este cu un perete static.

Fragment C2. Același indice identifică datele de poziție, mișcare și orientare.

```
  /* Vector state and cached collision bounds. */
  Vec3 *position_m{}, *velocity_mps{}, *angularVelocity_rad_s{};
  Vec3 *halfExtents_m{}, *inverseInertia_kg_m2{}, *force_N{}, *aabbExtent_m{};

  /* Rotation state. */
  Quat* orientation{};
```

Sursă C2 `src/shared/sim/state.h`
Linii 107–112

Capacitatea implicită este de 8 192 de objects, cu 128 de candidați pe object și $4{\cdot}128{+}6{\cdot}8 = 560$ de poziții în tabloul de contacte, pentru maximum patru puncte per pereche și opt colțuri pe fiecare față a incintei. Etapele din tabelul 1 sunt executate în aceeași ordine pe CPU și CUDA. Engine-ul raportează dacă lista de candidați este plină sau dacă *solver*-ul depășește limita de 256 de culori.

	Etapă măsurată	Operație
1	`applyForces`	Resetează marcajele de contact și aplică gravitația.
2	`integrateVelocity`	Integrează forța și amortizează vitezele.
3	`computeAabb`	Recalculează semidimensiunile în axele lumii.
4	`broadPhase`	Sortează cheile celulelor și caută perechi candidate.
5	`narrowPhase`	Testează perechile și construiește *contact manifolds*.
6	`generateContacts`	Adaugă contactele cu fețele incintei.
7	`solveContacts`	Colorează contactele, aplică impulsuri iterativ, înregistrează impacturile și corectează suprapunerile.
8	`integratePosition`	Avansează poziția și orientarea și aplică limitele incintei.
9	`updateSleep`	Actualizează starea de *sleep* și pune vitezele la zero pentru object-urile care intră în această stare.
10	`emitDiagnostics`	Raportează energie, cantitate de mișcare, contacte și corecții.

Tabelul 1: Ordinea etapelor unui pas de simulare.

2.2 Gravity, damping, and time integration

Pentru un object activ, gravitația este $\boldsymbol{F}_g = -mgs_g\boldsymbol{e}_y$, unde s_g este factorul gravitațional al object-ului. Din $m\,\mathrm{d}\boldsymbol{v}/\mathrm{d}t = \boldsymbol{F}_g$ rezultă o schimbare $-hgs_g$ a vitezei verticale înainte de amortizare. La $g = 9{,}81\,\mathrm{m/s^2}$, $s_g = 1$ și $h = 1/120\,\mathrm{s}$, aceasta este aproximativ $-0{,}082\,\mathrm{m/s}$ pe pas. Gravitația aplicată în centru are moment nul față de acesta. Rotația se modifică prin termenul de *angular damping* și prin impulsurile de contact.

Notăm coeficienții de *linear damping* și *angular damping* cu d_v și d_ω, ambii în $\mathrm{s^{-1}}$. Engine-ul actualizează vitezele astfel.

$$\boldsymbol{v}^{-} = \frac{\boldsymbol{v}_k + h\boldsymbol{F}_g/m}{1 + d_v h}, \qquad \boldsymbol{\omega}^{-} = \frac{\boldsymbol{\omega}_k}{1 + d_\omega h}. \tag{2}$$

Factorul $1/(1 + dh)$ este pasul implicit pentru termenul de amortizare, obținut din $u^{-} - u^{*} = -hd\,u^{-}$, unde u^{*} este valoarea înainte de amortizare. După ce solver-ul tratează contactele, engine-ul deplasează centrul cu $h\boldsymbol{v}^{+}$ de la poziția corectată. Folosirea vitezei noi pentru poziție este caracteristica metodei *semi-implicit Euler*.

Fragment C3. Adaugă accelerația și aplică amortizarea.

```
81  if (s.isSleeping[i]) return;
82
83  s.velocity_mps[i] += s.force_N[i] * (s.inverseMass_per_kg[i] * dt_s);
84  s.velocity_mps[i] = s.velocity_mps[i] / (1 + c.linearDamping_per_s * dt_s);
85  s.angularVelocity_rad_s[i] = s.angularVelocity_rad_s[i] / (1 + c.angularDamping_per_s *
        dt_s);
```

Sursă C3 `src/shared/sim/dynamics.h`
Linii 81–85

2.3 Quaternion rotation

Un *unit quaternion* $q = (\boldsymbol{q}, q_w)$ codifică rotația. Pentru axa unitară $\boldsymbol{a}$ și unghiul θ, $q = (\boldsymbol{a}\sin(\theta/2), \cos(\theta/2))$. Un vector local $\boldsymbol{r}$ se rotește folosind partea vectorială a produsului $q \otimes (\boldsymbol{r}, 0) \otimes q^*$, unde q^* este conjugatul. Pentru $p = (\boldsymbol{p}, p_w)$ și $q = (\boldsymbol{q}, q_w)$, *Hamilton product* se scrie

$$p \otimes q = \big(p_w\boldsymbol{q} + q_w\boldsymbol{p} + \boldsymbol{p} \times \boldsymbol{q},\ p_w q_w - \boldsymbol{p} \cdot \boldsymbol{q}\big). \tag{3}$$

Engine-ul exprimă *angular velocity* în *world space*, astfel încât incrementul de rotație se înmulțește din stânga. Pentru un pas mic, $\sin(\|\boldsymbol{\omega}^+\|h/2) \simeq \|\boldsymbol{\omega}^+\|h/2$. De aici rezultă actualizarea aproximativă

$$q_{k+1} = \text{normalize}\left(q_k + \tfrac{h}{2}(\boldsymbol{\omega}^+, 0) \otimes q_k\right). \tag{4}$$

Normalizarea împarte cele patru componente la norma lor. La 6 rad/s și 120 de pași pe secundă, object-ul se rotește cu aproximativ 0,05 rad, adică 2,9°, într-un pas.

Fragment C4. Calculează schimbarea orientării și normalizează quaternion-ul.

```
81  Quat d = Quat{omega_rad_s.x, omega_rad_s.y, omega_rad_s.z, 0} * q;
82  Real h = dt_s * Real(.5);
83  return normalize(Quat{q.x + d.x * h, q.y + d.y * h, q.z + d.z * h, q.w + d.w * h});
```

Sursă C4 `src/shared/sim/quaternion.h`
Linii 81–83

După integrare se recalculează limitele object-ului și se aplică proiecția în incintă. Implicit, un object aflat în contact poate intra în starea de *sleep* după 60 de pași sub pragurile de 0,05 m/s și 0,05 rad/s. Contactul cu un object activ care depășește un prag de mișcare îl poate reactiva.

2.4 Energy and diagnostics

Pentru un object, *Kinetic energy* include translația centrului și rotația. *Gravitational potential energy* are ca reper podeaua.

$$E_{\text{cin}} = \tfrac{1}{2}m\|\boldsymbol{v}\|^2 + \tfrac{1}{2}\boldsymbol{\omega} \cdot I_w\boldsymbol{\omega}, \tag{5}$$

$$E_{\text{pot}} = mgs_g(y - y_{\text{podea}}), \qquad \boldsymbol{P}_{\text{total}} = \sum_i m_i\boldsymbol{v}_i. \tag{6}$$

În cădere, energia potențială scade și viteza crește. Un impact oblic poate transfera energie de la translație la rotație. Frecarea, amortizarea și ricoșeurile inelastice pot reduce energia mecanică măsurată. Pereții exercită impulsuri externe asupra ansamblului de objects, deci cantitatea de mișcare totală a object-urilor se poate schimba. Diagnosticul mai numără perechile candidate, contactele și corecțiile la pereți și verifică valorile nefinite.

Fragment C5. Adaugă energiile și cantitatea de mișcare la total.

```
56  d.kineticEnergy_J += (mass * dot(s.velocity_mps[i], s.velocity_mps[i]) + angular) *
        HALF;
57  d.potentialEnergy_J += mass * c.gravity_mps2 * s.gravityScale_unitless[i] *
        (s.position_m[i].y + c.boxHalfExtents_m.y);
58  d.linearMomentum_kgmps += s.velocity_mps[i] * mass;
```

Sursă C5 `src/shared/sim/diagnostics.h`
Linii 56–58

3 Collision detection

3.1 Broad phase and spatial grid

Etapa *broad phase* restrânge căutarea la vecinătatea fiecărui object. Centrul $\boldsymbol{x}$ intră în celula $\boldsymbol{c} = \lfloor \boldsymbol{x}/L_{\text{cell}} \rfloor$, cu latura L_{cell} și rotunjire în jos pe fiecare axă. Cu $B = 2^{20}$, cheia din trei câmpuri de câte 21 de biți este

$$K_{\text{cel}} = 2^{42}(c_x + B) + 2^{21}(c_y + B) + (c_z + B) \tag{7}$$

pentru coordonate reprezentabile. Pe CPU se sortează perechile (cheie, indice). Pe CUDA, algoritmul *stable radix sort* păstrează ordinea indicilor pentru chei egale [8]. Astfel, cele două implementări vizitează object-urile dintr-o celulă în aceeași ordine.

BDEngine2D folosește o grilă cu *spatial hashing* pentru a reduce numărul de perechi de testat în 2D. În acest proiect, engine-ul folosește celule 3D și sortează cheile celulelor înainte de a căuta în vecinătatea fiecărui object [9].

Dacă R_i este *bounding radius* pentru object-ul i, iar $R_{\max}$ și $V_{\max}$ sunt cea mai mare rază, respectiv cea mai mare viteză din scenă, căutarea include s_i celule în fiecare sens pe fiecare axă. Cu *continuous collision detection* (CCD) activ, marja de deplasare este $D_i = h(\|\boldsymbol{v}_i^-\| + V_{\max})$ pentru căutarea în grilă și $D_{ij} = h(\|\boldsymbol{v}_i^-\| + \|\boldsymbol{v}_j^-\|)$ pentru perechea i, j. Cu CCD dezactivat, ambele marje sunt zero.

$$s_i = \max\left(1, \left\lceil \frac{R_i + R_{\max} + d_{\text{slop}} + D_i}{L_{\text{cell}}} \right\rceil\right). \tag{8}$$

Termenul de viteză provine din limita $\|\boldsymbol{v}_j^- - \boldsymbol{v}_i^-\| \le \|\boldsymbol{v}_i^-\| + \|\boldsymbol{v}_j^-\|$. Din fiecare celulă se reține doar candidații cu $j > i$, pentru a examina fiecare pereche o dată, și cu

$$\|\boldsymbol{x}_j - \boldsymbol{x}_i\|^2 \le (R_i + R_j + d_{\text{slop}} + D_{ij})^2 . \tag{9}$$

d_{slop} este toleranța *contact slop*. Cheile sortate permit localizarea începutului fiecărei celule prin *binary search*. În scene aglomerate, numărul de candidați crește chiar dacă se folosește grila.

Fragment C6. Mărește raza de căutare cu distanța maximă de deplasare când CCD este activ.

```
135  Real reach_m = s.boundingRadius_m[i] + maxRadius_m + c.contactSlop_m;
136  if (c.enableCcd) {
137      reach_m += (length(s.velocity_mps[i]) + maxSpeed_mps) * dt_s;
138  }
```

Sursă C6 `src/shared/sim/broad_phase.h`
Linii 135–138

3.2 Narrow phase and contact manifolds

Pentru două sfere, $\boldsymbol{d} = \boldsymbol{x}_b - \boldsymbol{x}_a$, $d_{\text{contact}} = r_a + r_b - \|\boldsymbol{d}\|$ și $\boldsymbol{n} = \boldsymbol{d}/\|\boldsymbol{d}\|$ când centrele sunt distincte. Două sfere cu raze de 0,5 m și centre la 0,9 m se suprapun cu 0,1 m. Punctul reprezentativ este $\boldsymbol{p} = \boldsymbol{x}_a + (r_a - d_{\text{contact}}/2)\boldsymbol{n}$.

Fragment C7. Calculează adâncimea din suma razelor și distanța centrelor.

```
35  Vec3 d = s.position_m[b] - s.position_m[a];
36  Vec3 normal = normalize(d);
37
38  Real distance = length(d);
39  Real r = s.halfExtents_m[a].x + s.halfExtents_m[b].x;
40  Real depth = r - distance;
```

Sursă C7 `src/shared/sim/contacts.h`
Linii 35–40

Pentru o sferă și un *oriented bounding box* (OBB), centrul sferei se transformă în axele cutiei, $\boldsymbol{z} = Q^{\mathsf{T}}(\boldsymbol{x}_{\text{sferă}} - \boldsymbol{x}_{\text{cutie}})$. Punctul cel mai apropiat are componentele $p_j = \text{clamp}(z_j, -b_j, b_j)$. În exterior, distanța $\|\boldsymbol{z} - \boldsymbol{p}\|$ se compară cu raza. În interior, normala se alege pe axa feței celei mai apropiate, determinată de $\min_j(b_j - |z_j|)$. În exterior, adâncimea este raza minus distanța până la punctul apropiat. În interior, este raza plus distanța până la fața aleasă. Punctul de contact în *world space* este $\boldsymbol{x}_{\text{cutie}} + Q\boldsymbol{p}$. Cilindrii și conurile folosesc aceeași aproximare geometrică prin cutie orientată. Implementarea perechilor sferă–sferă și sferă–cutie se găsește în `sphereSphere` și `sphereBox` (**C8**). În engine-ul 2D descris de Deepankar Agrawal, testul cerc–AABB folosește același pas de *clamp* pentru punctul cel mai apropiat. Aici calculul are loc în trei dimensiuni, în coordonatele locale ale OBB-ului [1].

Conform *Separating Axis Theorem* (SAT), două OBB-uri pot fi separate pe una dintre cele 15 axe candidate: trei normale ale fiecărei cutii și nouă produse vectoriale între direcțiile muchiilor [4]. Axele aproape nule sunt omise, iar celelalte sunt normalizate. Proiecția pe axa unitară $\boldsymbol{a}$ este un interval cu centru $\boldsymbol{a} \cdot \boldsymbol{x}_i$ și semilățime $r_{i,\text{proj}}$.

$$r_{i,\text{proj}}(\boldsymbol{a}) = \sum_{j=1}^{3} b_{ij} |\boldsymbol{a} \cdot \boldsymbol{u}_{ij}|, \tag{10}$$

$$d_{\text{contact}}(\boldsymbol{a}) = r_{a,\text{proj}}(\boldsymbol{a}) + r_{b,\text{proj}}(\boldsymbol{a}) - |(\boldsymbol{x}_b - \boldsymbol{x}_a) \cdot \boldsymbol{a}|. \tag{11}$$

Dacă $d_{\text{contact}} < -d_{\text{slop}}$ pe o axă, intervalele sunt separate. Altfel, engine-ul alege cea mai mică valoare a lui d_{contact} și orientează normala de la object-ul a spre object-ul b.

Fragment C9. Respinge perechea când proiecțiile sunt separate peste toleranță.

```
180  Real overlap = projectRadius(s, a, axis) + projectRadius(s, b, axis) -
         absolute(dot(delta, axis));
181
182  if (overlap < -c.contactSlop_m) {
183      return false;
184  }
```

Sursă C9 `src/shared/sim/contacts.h`
Linii 180–184

Un contact între două fețe poate fi reprezentat prin mai multe puncte. Când normala se aliniază cu o față, engine-ul o alege ca *reference face* și aplică *polygon clipping* pe *incident face*, folosind cele patru semiplane laterale. Pentru o muchie cu distanțe cu semn d_0, d_1 de semne opuse, punctul de intersecție este $\boldsymbol{p}_0 + \frac{d_0}{d_0 - d_1}(\boldsymbol{p}_1 - \boldsymbol{p}_0)$. Rezultă până la patru puncte distincte, adică un *contact manifold*. Când decuparea nu produce puncte utile, rămâne contactul reprezentativ dintre punctele de sprijin. Perechile cu o sferă produc un singur contact.

Fragment C10. Aplică polygon clipping și limitează numărul de puncte.

```
204  count = clipPolygon(first, count, second, axes[ru], center, h[ru]);
205  count = clipPolygon(second, count, first, -axes[ru], center, h[ru]);
206  count = clipPolygon(first, count, second, axes[rv], center, h[rv]);
207  count = clipPolygon(second, count, first, -axes[rv], center, h[rv]);
208
209  int written = 0;
210
211  // Limit to max 4 manifold points for stability and performance.
212  int samples = count > 4 ? 4 : count;
```

Sursă C10 `src/shared/sim/manifolds.h`
Linii 204–212

3.3 Intersection tests

Un *axis-aligned bounding box* (AABB) se suprapune cu altul atunci când intervalele lor se suprapun pe toate cele trei axe. Pentru o cutie orientată cu matrice Q, semidimensiunile AABB sunt $\boldsymbol{b}_{\mathrm{AABB}} = |Q|\boldsymbol{b}$, cu modul aplicat element cu element. Grila folosește *bounding radius*, iar contactele dintre objects folosesc sferele și OBB-urile descrise mai sus. Cele două funcții de mai jos sunt teste geometrice separate, verificate prin *unit tests*.

`rayTriangle`. Funcția primește originea $\boldsymbol{o}$, direcția $\boldsymbol{d}$ și vârfurile triunghiului $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$. Folosește algoritmul Möller–Trumbore pentru a rezolva ecuația [7].

$$\boldsymbol{o} + t\boldsymbol{d} = \boldsymbol{a} + u(\boldsymbol{b} - \boldsymbol{a}) + v(\boldsymbol{c} - \boldsymbol{a}). \tag{12}$$

Ponderile vârfurilor sunt $1 - u - v$, u și v. Dacă una dintre ele e negativă, punctul se află în afara triunghiului. Funcția acceptă intersecția pe oricare dintre fețe, inclusiv la muchii și vârfuri. Respinge cazurile în care determinantul este aproape zero, cum se întâmplă când raza este paralelă cu planul. La succes întoarce `true` și scrie parametrul t în `distance_m`. Acesta este distanța în metri dacă $\|\boldsymbol{d}\| = 1$. Parametrul t trebuie să fie nenegativ pentru ca punctul să se afle pe semidreapta dată. Pentru triunghiul cu vârfurile $(0, 0, 0)$, $(1, 0, 0)$ și $(0, 1, 0)$, raza din (0,2; 0,2; 1) cu direcția $(0, 0, -1)$ atinge triunghiul la $t = 1$. Aceeași direcție din $(2, 2, 1)$ intersectează planul în afara triunghiului. Implementarea de ray tracing în C# a lui Emre Safa Baltacı folosește același test Möller–Trumbore [2]. **C11**

`triangleTriangle`. Funcția primește două triunghiuri în același sistem de coordonate și întoarce `true` dacă testul SAT găsește suprapunere. Proiectează vârfurile pe normalele celor două fețe, pe produsele vectoriale dintre perechi de muchii și pe axele din plan construite din normala și muchiile fiecărui triunghi. O singură axă cu intervale disjuncte este suficientă pentru `false`. Axele din plan separă triunghiurile coplanare când proiecția pe normala comună coincide. Funcția ignoră axele aproape nule și respinge triunghiurile degenerate, cu arie aproape zero. Returnează un `bool` pentru suprapunere.

De exemplu, triunghiul cu vârfurile $(2, 2, 0)$, $(3, 2, 0)$ și $(2, 3, 0)$ este coplanar cu triunghiul din exemplul de mai sus, dar proiecțiile pe axa x sunt $[0, 1]$ și $[2, 3]$. Testul întoarce `false`. Testele proiectului acoperă și intersecția necoplanară, includerea unui triunghi coplanar și respingerea unui triunghi degenerat. **C12 C13**

3.4 Continuous collision detection and wall contacts

Între două instantanee, un object rapid poate trece dincolo de un obstacol subțire. Pentru două sfere separate, engine-ul caută *time of impact*, când centrele ajung la distanța $r = r_a + r_b$, cu

viteza relativă $\boldsymbol{w} = \boldsymbol{v}_b^- - \boldsymbol{v}_a^-$. Din $\|\boldsymbol{d} + t\boldsymbol{w}\|^2 = r^2$ rezultă

$$A = \boldsymbol{w}\cdot\boldsymbol{w}, \qquad B = \boldsymbol{d}\cdot\boldsymbol{w}, \qquad C = \boldsymbol{d}\cdot\boldsymbol{d} - r^2, \tag{13}$$

$$At^2 + 2Bt + C = 0, \qquad t_\star = \frac{-B - \sqrt{B^2 - AC}}{A}. \tag{14}$$

Engine-ul testează dacă perechea se apropie, dacă discriminantul este nenegativ și dacă $t_\star \in [0, h]$. Dacă da, folosește normala la momentul estimat și generează contactul cu adâncime zero pentru pasul curent.

Fragment C14. Acceptă impactul estimat dacă are loc în pasul curent.

```
54  Real impact_s = (-bb - Real(sqrt(discriminant))) / aa;
55  if (impact_s < 0 || impact_s > dt_s) {
56      return false;
57  }
58
59  normal = normalize(d + relative * impact_s);
60  depth = 0;
```

Sursă C14 `src/shared/sim/contacts.h`
Linii 54–60

Pentru o față a incintei cu normala exterioară $\boldsymbol{n}$ și distanța H_f față de origine, un punct $\boldsymbol{p}$ al object-ului are adâncimea $d_f = \boldsymbol{n}\cdot\boldsymbol{p} - H_f$. Viteza punctului spre exterior este $u_f = \boldsymbol{n}\cdot(\boldsymbol{v} + \boldsymbol{\omega}\times(\boldsymbol{p} - \boldsymbol{x}))$. Se folosește punctul de sprijin al sferei sau fiecare dintre cele opt colțuri ale cutiei. CCD păstrează și *speculative contacts* dacă $d_f + hu_f$ prezice traversarea. Pentru cutii se adaugă marja $R\|\boldsymbol{\omega}\|^2 h^2$ și se verifică separat limita conservatoare bazată pe raza de încadrare și deplasarea maximă. Perechile sferă–OBB și OBB–OBB dintre objects sunt testate la pozițiile curente, astfel încât verificarea continuă este parțială. Logica pereților se află în `wallContact` și `wallVertexContact` (**C15**).

4 Iterative contact solver

4.1 Relative velocity and effective mass

Considerăm un contact în punctul $\boldsymbol{p}$, cu *unit normal* $\boldsymbol{n}$ dinspre a spre b. Cu $\boldsymbol{r}_i = \boldsymbol{p} - \boldsymbol{x}_i$, viteza suprafeței object-ului i în punctul de contact este $\boldsymbol{v}_i + \boldsymbol{\omega}_i \times \boldsymbol{r}_i$. Diferența dintre cele două viteze este *relative velocity*,

$$\boldsymbol{u} = (\boldsymbol{v}_b + \boldsymbol{\omega}_b\times\boldsymbol{r}_b) - (\boldsymbol{v}_a + \boldsymbol{\omega}_a\times\boldsymbol{r}_a), \qquad u_n = \boldsymbol{u}\cdot\boldsymbol{n}. \tag{15}$$

Dacă $u_n < 0$, suprafețele se apropie. Un perete static are viteza, masa inversă și inerția inversă egale cu zero în calcul.

Un impuls scalar J pe direcția unitară $\boldsymbol{d}$ schimbă viteza relativă proiectată pe acea direcție cu $JK(\boldsymbol{d})$, unde K este *inverse effective mass*.

$$K(\boldsymbol{d}) = \sum_{i\in\{a,b\}} \left[m_i^{-1} + (\boldsymbol{r}_i\times\boldsymbol{d})\cdot I_{i,w}^{-1}(\boldsymbol{r}_i\times\boldsymbol{d}) \right]. \tag{16}$$

Primul termen vine din translația centrului. Al doilea vine din schimbarea vitezei unghiulare produsă de impuls. Într-adevăr, $\boldsymbol{d}\cdot(\Delta\boldsymbol{\omega}_i\times\boldsymbol{r}_i) = \Delta\boldsymbol{\omega}_i\cdot(\boldsymbol{r}_i\times\boldsymbol{d})$. Dacă linia de acțiune trece prin centru, produsul vectorial este zero și object-ul contribuie doar prin m_i^{-1}. Într-o lovitură frontală între două sfere, *effective mass* coincide cu *reduced mass* $m_a m_b/(m_a + m_b)$.

Fragment C16. Contribuția unui object la $K(\boldsymbol{d})$ include masa inversă și termenul de rotație.

```
27  Vec3 ra = ct.point_m - s.position_m[ct.a_i];
28
29  // Linear inverse mass plus the angular contribution (r x axis) dot (I^-1 * (r x axis)).
30  Real result = s.inverseMass_per_kg[ct.a_i] + dot(axis, cross(inverseInertia(s, ct.a_i,
        cross(ra, axis)), ra));
```

Sursă C16 `src/shared/sim/solver.h`
Linii 27–30

4.2 Restitution and normal impulse

La începutul pasului se calculează valoarea *target velocity* $u_\star$. Dacă viteza normală inițială $u_{n,0}$ este sub pragul negativ de ricoșeu, ținta este $-e\,u_{n,0}$, unde e este *coefficient of restitution*. Altfel, ținta este zero. Pentru un *speculative contact* cu adâncimea $d_{\text{contact}} < -d_{\text{slop}}$, solver-ul ia minimul dintre această țintă și d_{contact}/h. Această limită permite apropierea cu cel mult distanța disponibilă într-un pas.

La fiecare iterație, solver-ul ajustează valoarea *accumulated normal impulse* [3].

$$J_n' = \max\!\left(0,\, J_n + \frac{u_\star - u_n}{K(\boldsymbol{n})}\right), \qquad \Delta\boldsymbol{J}_n = (J_n' - J_n)\boldsymbol{n}. \tag{17}$$

Proiecția pe valori nenegative permite contactului să împingă. Incrementul poate avea semn negativ dacă o iterație anterioară a aplicat un impuls prea mare. Object-ul a primește $-\Delta\boldsymbol{J}_n$, iar object-ul b primește $+\Delta\boldsymbol{J}_n$. Impulsurile se acumulează între iterațiile aceluiași pas și pornesc de la zero pentru contactele nou generate.

Pentru o comparație cu un alt engine, Maciej Mizerski descrie aplicarea impulsurilor în spațiul vitezelor și folosirea *Projected Gauss–Seidel* în Roblox. Același principiu apare aici în actualizarea repetată a impulsului de contact. Ordinea pe culori și execuția CUDA sunt detalii ale acestui proiect [6].

Fragment C17. Variabila `massInv` este $K(\boldsymbol{n})$. Se aplică diferența de impuls.

```
138  Real increment = (ct.targetVelocity_mps - dot(relative, ct.normal)) / massInv;
139  Real previous = ct.normalImpulse_Ns;
140
141  // Clamp the accumulated impulse to be non-negative (unilateral constraint).
142  ct.normalImpulse_Ns = maximum(0, previous + increment);
143
144  // Apply the normal impulse change.
145  applyImpulse(s, ct, ct.normal * (ct.normalImpulse_Ns - previous));
```

Sursă C17 `src/shared/sim/solver.h`
Linii 138–145

Exemplu între două sfere. Sfera a, de $1\,\text{kg}$, ajunge cu $2\,\text{m/s}$ la sfera b, care are aceeași masă și stă pe loc. Pentru o lovitură pe linia centrelor, $K(\boldsymbol{n}) = 2\,\text{kg}^{-1}$. Cu $e = 0{,}5$, viteza de separare dorită este $1\,\text{m/s}$. Impulsul calculat este $(1 - (-2))/2 = 1{,}5\,\text{N s}$. Vitezele finale sunt $0{,}5\,\text{m/s}$ și $1{,}5\,\text{m/s}$ în direcția inițială a sferei a. Cantitatea de mișcare totală rămâne $2\,\text{kg m/s}$, iar energia cinetică scade de la $2\,\text{J}$ la $1{,}25\,\text{J}$. Pentru contactul izolat, fără frecare, energia pe direcția normală scade de la E_n la $e^2 E_n$. Diferența este $(1 - e^2)E_n$.

Exemplu de cădere și ricoșeu. În modelul ideal fără amortizare, o bilă lăsată de la înălțimea H ajunge la podea cu viteza $\sqrt{2gH}$, din egalitatea $mgH = \tfrac{1}{2}mv^2$. După o lovitură verticală, pleacă în sus cu viteza $e\sqrt{2gH}$ și atinge înălțimea $e^2 H$. O cădere de un metru cu $e = 0{,}5$ produce

un ricoșeu de aproximativ 25 cm. Pasul finit și amortizarea pot schimba această valoare în simulator. Pragul de viteză pentru *restitution* ajută contactele aproape în repaus să se stabilizeze.

4.3 Angular impulse and friction

Dacă $\Delta \boldsymbol{J}$ este impulsul cu semn primit de un object,

$$\Delta \boldsymbol{P} = \Delta \boldsymbol{J}, \qquad \Delta \boldsymbol{L} = \boldsymbol{r} \times \Delta \boldsymbol{J}, \qquad \Delta \boldsymbol{\omega} = I_w^{-1} \Delta \boldsymbol{L}. \tag{18}$$

Modulul *angular impulse* este $\|\boldsymbol{r}\| \|\Delta \boldsymbol{J}\| \sin\theta$, unde θ este unghiul dintre braț și impuls. Un impuls de $2\,\mathrm{N\,s}$ aplicat perpendicular la $0{,}3\,\mathrm{m}$ de centru produce $0{,}6\,\mathrm{kg\,m^2/s}$ moment cinetic. Dacă inerția pe axa respectivă este $0{,}2\,\mathrm{kg\,m^2}$, viteza unghiulară se schimbă cu $3\,\mathrm{rad/s}$. Engine-ul folosește viteza actualizată la integrarea orientării prin *quaternion rotation*.

Fragment C18. Actualizează vitezele object-ului A. Object-ul B primește impulsul cu semnul opus.

```
53  // Update body A (subtract impulse).
54  s.velocity_mps[a] -= impulse_Ns * s.inverseMass_per_kg[a];
55  s.angularVelocity_rad_s[a] -= inverseInertia(s, a, cross(ct.point_m - s.position_m[a],
        impulse_Ns));
```

Sursă C18 `src/shared/sim/solver.h`
Linii 53–55

După impulsul normal se recalculează viteza relativă și se extrage alunecarea $\boldsymbol{u}_t = \boldsymbol{u} - (\boldsymbol{u}\cdot\boldsymbol{n})\boldsymbol{n}$. Pentru $\|\boldsymbol{u}_t\| > 0$, se alege direcția $\boldsymbol{t} = \boldsymbol{u}_t / \|\boldsymbol{u}_t\|$. Valoarea propusă pentru *tangential impulse* este

$$\boldsymbol{J}_t^{\mathrm{cand}} = \boldsymbol{J}_t - \frac{\boldsymbol{u}_t}{K(\boldsymbol{t})}. \tag{19}$$

Dacă modulul impulsului propus este cel mult $\mu_s J_n'$, solver-ul îl acceptă. Dacă limita este depășită, solver-ul păstrează direcția impulsului și îi reduce modulul la $\mu_d J_n'$. Se aplică diferența față de impulsul tangențial deja acumulat. Cu $J_n' = 2\,\mathrm{N\,s}$, $\mu_s = 0{,}5$ și $\mu_d = 0{,}4$, limita pentru *static friction* este $1\,\mathrm{N\,s}$. Dacă sunt necesari $1{,}4\,\mathrm{N\,s}$ pentru oprirea alunecării, limita pentru *kinetic friction* este $0{,}8\,\mathrm{N\,s}$.

Fragment C19. Verifică limita statică și aplică limita cinetică dacă este depășită.

```
165  Vec3 proposed = ct.tangentImpulse_Ns - tangent * (speed / tangentMass);
166  Real magnitude = length(proposed);
167
168  // Apply Coulomb friction limit.
169  if (magnitude > c.frictionStatic_unitless * ct.normalImpulse_Ns) {
170      // If static friction is exceeded, clamp to dynamic friction.
171      proposed = normalize(proposed) * (c.frictionDynamic_unitless * ct.normalImpulse_Ns);
172  }
```

Sursă C19 `src/shared/sim/solver.h`
Linii 165–172

Pe o bilă care alunecă pe podea, *friction* acționează în punctul de contact. Impulsul tangențial reduce alunecarea și poate crește viteza unghiulară în sensul rulării. La *rolling without slipping* pe un plan static, $\boldsymbol{v} + \boldsymbol{\omega} \times \boldsymbol{r}$ are componenta tangențială nulă la contact. Pentru o bilă cu axa de rotație perpendiculară pe direcția deplasării, aceasta dă $\|\boldsymbol{v}\| = r\|\boldsymbol{\omega}\|$.

4.4 Position correction

Etapa *position correction* se aplică separat de impulsuri. Pentru $w_i = m_i^{-1}$, $d_{\text{excess}} = \max(0, d_{\text{contact}} - d_{\text{slop}})$, coeficientul de corecție k_{corr} și factorul s_m, deplasările sunt

$$\Delta \boldsymbol{x}_a = -\frac{w_a}{w_a + w_b} k_{\text{corr}} s_m d_{\text{excess}} \boldsymbol{n}, \qquad \Delta \boldsymbol{x}_b = +\frac{w_b}{w_a + w_b} k_{\text{corr}} s_m d_{\text{excess}} \boldsymbol{n}. \tag{20}$$

Object-ul cu masă mai mică se deplasează mai mult. Factorul $s_m = 1/4$ la un contact OBB–OBB limitează corecția cumulată a unui *contact manifold* cu mai multe puncte. Pentru un contact cu sferă se folosește 1. Contactele cutie–perete au o corecție separată pe baza dimensiunilor cutiei. După avansarea poziției și orientării, o proiecție finală limitează centrul în funcție de semidimensiunile actualizate. Valorile implicite sunt $d_{\text{slop}} = 0{,}001\,\text{m}$ și $k_{\text{corr}} = 0{,}6$. Sursa formulei este `correctContactPosition` (**C20**).

5 CUDA execution and reproducibility

Fiecare contact poate fi privit ca o muchie între object-urile pe care le modifică. Contactele cu pereții au un singur object activ. Algoritmul de *graph coloring* parcurge contactele într-o ordine fixă și alege pentru fiecare prima culoare liberă la object-urile implicate. Contactele de aceeași culoare pot fi procesate în paralel, fiindcă fiecare contact modifică alte objects. Culorile următoare văd vitezele deja actualizate, ca într-o iterație Gauss–Seidel pe grupuri.

Pe CUDA, un singur *thread* colorează contactele, iar solver-ul rulează într-un *thread block* cu 128 de threads. Fiecare thread procesează o parte dintre contactele unei culori. Sincronizarea prin `__syncthreads()` are loc înainte de următoarea culoare. CPU folosește aceeași ordine în mod serial. Puține contacte într-o culoare lasă threads inactive, iar mai multe culori înseamnă mai multe *synchronization barriers*. Un singur bloc limitează solver-ul la un *streaming multiprocessor* (SM). Etapa *broad phase*, generarea contactelor și alte etape folosesc mai multe blocuri. Maximele globale pentru căutare, colorarea și diagnosticul rulează serial pe dispozitiv.

Fragment C21. Thread-urile tratează contactele aceleiași culori în paralel și se sincronizează înainte de culoarea următoare.

```
68  for (int iteration = 0; iteration < c.solverIterations_i; ++iteration) {
69      for (int color = 0; color < *number; ++color) {
70          for (int k = offsets[color] + int(threadIdx.x); k < offsets[color + 1]; k +=
        int(blockDim.x)) {
71              solveContactConstraint(s, contacts[indices[k]], c);
72          }
73          __syncthreads();
74      }
75  }
```

Sursă C21 `src/physics/backend/cuda/solver.cu`
Linii 68–75

CUDA backend-ul păstrează tablourile de stare pe GPU și folosește spațiu CUB prealocat. Transferurile trec prin *pinned host memory*. Copiile sunt programate asincron pe același *CUDA stream*, cu sincronizare la limita apelului. Pasul sincronizează și rezultatele de diagnostic. Viewer-ul și executabilul de test cer separat descărcarea stării. Timpii etapelor pe GPU se măsoară cu *CUDA events*, iar pe CPU cu un *monotonic clock*. Timpul total include lansarea operațiilor CUDA, copierea datelor de diagnostic și sincronizarea de la sfârșitul pasului.

Ordinea perechilor și a culorilor este deterministă. Opțiunile `/fp:strict` sau `-ffp-contract=off` pe CPU și `--fmad=false` pe CUDA reduc diferențele aritmetice. Executabilul salvează la fiecare pas un hash FNV-1a de 64 de biți pentru numărul de objects, pas și câmpurile selectate ale stării, inclusiv poziții, orientări, viteze și starea de *sleep*. La *replay*, engine-ul încarcă starea inițială și

configurația, apoi compară fiecare pas cu *trace*-ul salvat. Verificarea acoperă câmpurile incluse în hash, pe scenele și configurația testate.

6 Scene rendering and impact visualization

6.1 From physics state to rendered frame

Viewer-ul primește starea după pasul de simulare. Pentru fiecare formă refolosește același *mesh* și îl desenează pentru mai multe objects prin *instanced rendering*. Fiecare object are propria *model matrix* $M = TQS$. Matricea M aplică fiecărui vârf local transformările *scale*, *rotation* și *translation*. Rotația vine din quaternion-ul object-ului, iar translația din poziția centrului. Programul *vertex shader* aplică apoi matricea de vizualizare V și proiecția camerei P, în ordinea $\boldsymbol{x}_{\text{clip}} = PVM\boldsymbol{x}_{\text{local}}$.

Fragment C22. Construiește transformarea object-ului prin scalare, rotație și translație.

```
116  // Column vectors apply scale first, then rotation, then translation into world space.
117  const auto model =
118      glm::translate(glm::mat4(1.0f), glm::vec3(position_m.x, position_m.y,
         position_m.z)) *
119      glm::mat4_cast(glm::quat(orientation.w, orientation.x, orientation.y,
         orientation.z)) *
120      glm::scale(glm::mat4(1.0f), glm::vec3(halfExtents_m.x, halfExtents_m.y,
         halfExtents_m.z));
```

Sursă C22 `src/client/render_gl/rigid_bodies/draw_objects.cpp`
Linii 116–120

Normala descrie orientarea unei suprafețe. La scalare neuniformă, ea se transformă prin inversa transpusă a părții liniare a matricei de model, pentru a rămâne perpendiculară pe suprafață. Programul *fragment shader* combină culoarea de bază cu lumina principală, lumina de umplere, lumina de contur și lumina ambientală. Termenul de *diffuse lighting* este proporțional cu $\max(0, \boldsymbol{n}\cdot\boldsymbol{l})$, unde $\boldsymbol{l}$ indică direcția luminii. Termenul *specular highlight* dă luciul. Culorile sunt comprimate prin *ACES tone mapping* și convertite în sRGB înainte de suprapunerea urmelor. Pereții transparenți sunt desenați de la cel mai îndepărtat la cel mai apropiat, după object-urile opace.

6.2 Soft contact shadows

Pentru un object cu raza de încadrare R și semidimensiunea verticală AABB b_y, distanța estimată până la podea este $a = y - y_{\text{podea}} - b_y$. Engine-ul plasează o umbră sub centrul object-ului, la 3 mm deasupra podelei, cu

$$F_{\text{shadow}} = \operatorname{clamp}\Big(1 - \frac{a}{3R}, 0, 1\Big). \tag{21}$$

Umbra este mai intensă lângă podea. În timpul unui ricoșeu, a crește și umbra se estompează până dispare. Calculul se află în `buildShadows` (**C23**).

Shader-ul evaluează mai multe eșantioane dispuse pe o *golden-angle spiral*. Pentru coordonata locală $\boldsymbol{z}$ din umbră și deplasările $\boldsymbol{t}_j$ ale eșantioanelor, opacitatea este proporțională cu $F_{\text{shadow}}\, N^{-1} \sum_j \exp(-k_{\text{shadow}}\|\boldsymbol{z}+\boldsymbol{t}_j\|^2)$. Media netezește marginea. Podeaua se desenează prima, astfel încât umbra să întunece grila înainte de desenarea object-urilor.

Fragment C24. Calculează opacitatea umbrei folosind factorul de fade din `visual.z`.

```
    // Accumulate the exponential falloff based on the distance to the sample.
    sum += exp(-dot(p + tap, p + tap) * WORLD.SHADOW.SHADOW_FOOTPRINT_FALLOFF);
}

// Output the tint with the averaged shadow density as alpha.
fragmentColor = vec4(
    WORLD.SHADOW.SHADOW_TINT,
    WORLD.SHADOW.SHADOW_FOOTPRINT_STRENGTH * visual.z * sum /
    float(WORLD.SHADOW.SHADOW_TAP_COUNT)
);
```

Sursă C24 `src/client/render_gl/shaders/shadow.frag`
Linii 41–49

6.3 Impact energy

Înainte de rezolvare se măsoară componenta normală a vitezei de apropiere. Folosind *effective mass* $1/K(\boldsymbol{n})$,

$$v_c = \max(0, -\boldsymbol{u}_0 \cdot \boldsymbol{n}), \qquad E_n = \tfrac{1}{2}\big(1/K(\boldsymbol{n})\big)v_c^2. \tag{22}$$

E_n reprezintă energia de apropiere a unui contact pe direcția normală. La aceeași masă efectivă, dublarea vitezei înmulțește energia cu patru. Cele trei sfere din figura 2 pornesc cu 1, 4 și 9 m/s, cu gravitația și amortizarea dezactivate. Scena este definită de `viewerScenario` și `populateViewer` (**C25**).

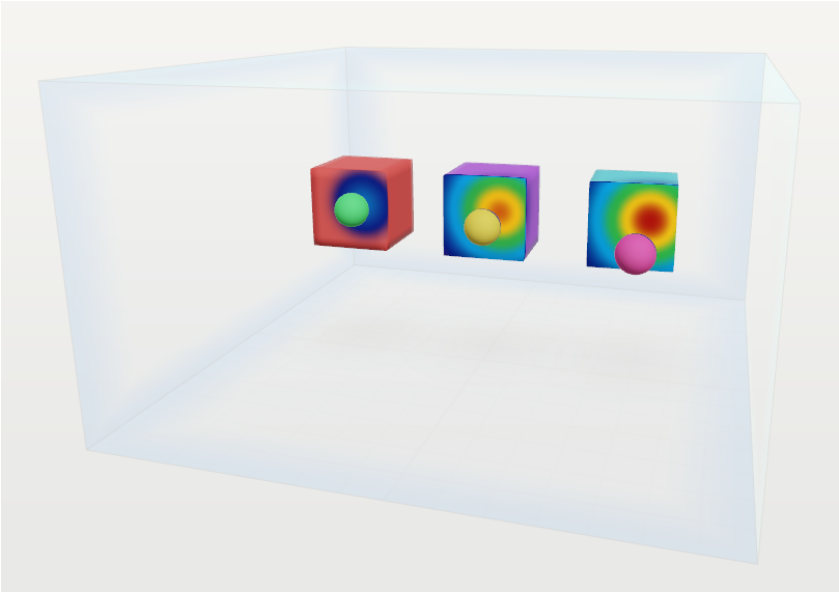


Figura 2: Urme de impact pe cele trei ținte. Albastrul și verdele corespund intensităților mai mici, iar galbenul și roșul celor mai mari.
Imagine din proiect `media/cuda-impact.png`.

Fragment C26. Calculează energia normală din viteza de apropiere și masa efectivă inversă.

```
60  Real speed = maximum(0,  -dot(contactVelocity(s, ct.b_i, ct.point_m) -
        contactVelocity(s, ct.a_i, ct.point_m), ct.normal));
61
62  // Calculate the inverse of the effective mass along the normal.
63  Real inverse = effectiveMassInverse(s, ct, ct.normal);
64  return {speed, /* E = 0.5 * v^2 / m_inv */ inverse >
        settings::Collisions::Geometry::EPSILON ? HALF * speed * speed / inverse : 0};
```

Sursă C26 `src/shared/sim/impacts.h`
Linii 60–64

După rezolvare sunt păstrate evenimentele cu $v_c \geq 0{,}5\,\mathrm{m/s}$, $E_n \geq 0{,}005\,\mathrm{J}$ și impuls normal acumulat peste $10^{-4}\,\mathrm{N\,s}$. Fiecare object păstrează până la patru evenimente pe pas, cu prioritate pentru cele cu energie mare.

6.4 Collision heatmap

Viewer-ul păstrează până la opt *impact patches* pe object sau pe o față a incintei. Punctul și normala sunt stocate în axele locale ale object-ului, astfel încât urma se rotește odată cu acesta. Pereții folosesc puncte proiectate pe suprafață. Pentru energia E exprimată numeric în jouli, pragul de saturație E_{red}, vârsta t și durata T, intensitatea urmei este

$$H(E,t) = \mathrm{clamp}\!\left(\frac{\ln(1+E)}{\ln(1+E_{\mathrm{red}})}, 0, 1\right) \max(0, 1 - t/T)^{0{,}6}. \tag{23}$$

Cu pragul de 10 J, o urmă nouă de 1 J are $\ln 2 / \ln 11 \approx 0{,}29$, iar una de 10 J pornește de la 1. Logaritmul face vizibile și loviturile slabe. Urmele mai intense primesc raze mai mari, în limitele stabilite de dimensiunea object-ului. Datele sunt pregătite de `ImpactHistory::strength` și `ImpactHistory::advance` (**C27**) și transferate într-un *shader storage buffer object* (SSBO).

Pentru un eșantion de suprafață în punctul local $\boldsymbol{x}$ cu normala $\boldsymbol{n}$, pata cu centrul $\boldsymbol{p}$, raza r_p și normala $\boldsymbol{n}_p$, contribuția unui *impact patch* este

$$H(E,t)\exp\!\left(-k\,\frac{\|\boldsymbol{x}-\boldsymbol{p}\|^2}{r_p^2}\right)\mathrm{smoothstep}(a,b,\boldsymbol{n}\cdot\boldsymbol{n}_p). \tag{24}$$

Termenul de *distance falloff* reduce intensitatea odată cu distanța. Termenul de *normal facing* reduce culoarea pe fețele întoarse în altă direcție. Funcția GLSL `smoothstep(a, b, u)` transformă valoarea u într-o pondere între 0 și 1, cu pragurile $a < b$. Întoarce 0 până la a și 1 de la b în sus. Între praguri folosește o interpolare cubică Hermite [5].

$$z = \mathrm{clamp}\!\left(\frac{u-a}{b-a}, 0, 1\right), \qquad \mathrm{smoothstep}(a,b,u) = 3z^2 - 2z^3. \tag{25}$$

Panta în raport cu z este $6z(1-z)$, deci trecerea începe și se termină cu pantă zero. De exemplu, la un sfert din interval ponderea este 0,15625, iar la trei sferturi este 0,84375. În calculul *normal facing*, $u = \boldsymbol{n}\cdot\boldsymbol{n}_p$ măsoară alinierea normalelor. Ponderea crește gradual când suprafața se aliniază cu urma, astfel încât culoarea se estompează la margine în loc să se schimbe brusc. Fragment shader-ul păstrează intensitatea maximă dintre urmele suprapuse.

Fragment C28. Combină distance falloff cu orientarea suprafeței pentru a calcula intensitatea urmei.

```
66  vec3 delta = (local - pointRadius.xyz) / pointRadius.w;
67
68  float facing = smoothstep(WORLD.IMPACTS.IMPACT_FACING_LO,
        WORLD.IMPACTS.IMPACT_FACING_HI, dot(n, normalStrength.xyz));
69
70  return normalStrength.w * exp(-WORLD.IMPACTS.IMPACT_FALLOFF * dot(delta, delta)) *
        facing;
```

Sursă C28 `src/client/render_gl/shaders/common/impact.glsl`
Linii 66–70

Paleta împarte intervalul $[0, 1]$ în patru segmente între albastru, cyan, verde, galben și roșu. În fiecare segment, `smoothstep(0.0, 1.0, u)` remapează poziția locală u, iar `mix` folosește ponderea rezultată pentru a combina cele două culori de la capetele segmentului. Centrul unei urme proaspete și saturate poate fi roșu, în timp ce marginile aceleiași urme trec prin celelalte culori. Shaderul amestecă paleta cu culoarea iluminată a object-ului. *Heatmap*-ul reprezintă severitatea impactului și estomparea urmei pe suprafață.

Fragment C29. Interpolează între culorile vecine ale paletei.

```
41  vec3 impactPalette(float heat) {
42      float ramp = clamp(heat, 0.0, 1.0) * 4.0;
43      int i = min(int(ramp), 3);
44      return mix(impactStop(i), impactStop(i + 1), smoothstep(0.0, 1.0, ramp - float(i)));
45  }
```

Sursă C29 `src/client/render_gl/shaders/common/impact.glsl`
Linii 41–45

Fragment C30. Suprapune culoarea de impact peste culoarea iluminată.

```
141  float heat = impactHeatAll(impactIndex, impactLocalPoint_m, impactLocalNormal,
         uImpactEnabled);
142  float key = max(dot(N, L_key), 0.0);
143
144  // Apply ACES tonemapping -> sRGB encoding -> impact heat -> final output.
145  fragmentColor = vec4(applyImpact(srgb(aces(result)), heat, key), 1.0);
```

Sursă C30 `src/client/render_gl/shaders/body.frag`
Linii 141–145

7 Interfața viewer-ului

Viewer-ul folosește Dear ImGui pentru *widgets* și afișează scena în același frame cu panourile din figura 3. Butonul *Widgets* comută vizibilitatea panourilor și oferă control pentru dimensiunea textului. Tasta `F1` ascunde sau afișează interfața.

Simulation. *Scenario* încarcă una dintre scene, iar *Seed* stabilește *random seed*-ul folosit la inițializare. *Run* și *Pause* controlează avansarea simulării, iar *Reset* recreează scena. Schimbarea din *Backend* copiază state-ul curent în noul backend, astfel încât poți continua aceeași simulare pe CPU sau CUDA.

Spawn objects. Alegi *Shape* și *Count*, apoi *Spawn* adaugă objects în scena curentă. Dacă engine-ul ajunge la limita de capacitate sau nu găsește o poziție validă pentru următorul object, se oprește și scrie numărul adăugat în log.

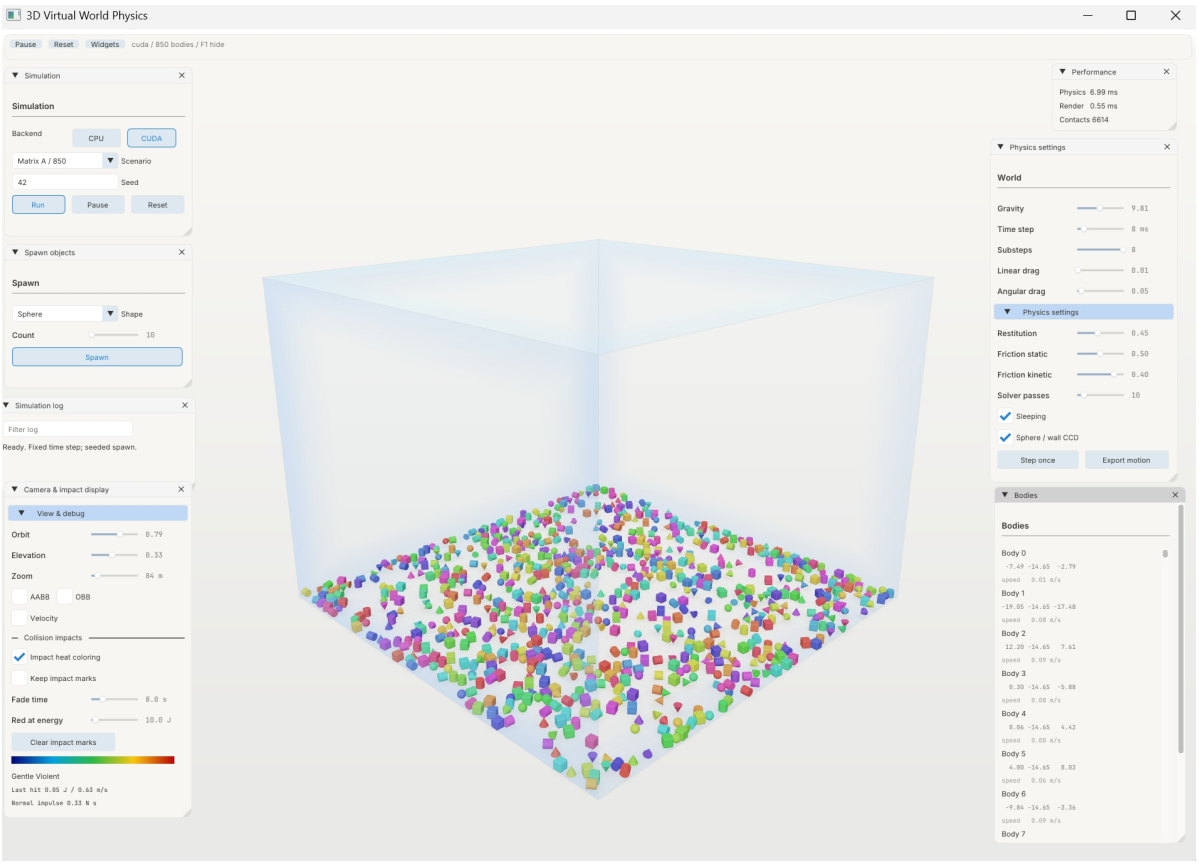


Figura 3: Viewer-ul cu scena în centru, comenzile în stânga și panourile de performanță și inspecție în dreapta.

Imagine din proiect `docs/figures/app_running.jpg`.

Physics settings. *Gravity*, *Linear drag*, *Angular drag*, *Restitution*, *Friction static* și *Friction kinetic* controlează dinamica. *Time step* este durata unui pas de simulare. *Substeps* limitează câți pași se pot executa într-un frame, iar *Solver passes* stabilește câte iterații de contact se execută în fiecare pas. *Sleeping* și *Sphere / wall CCD* activează comportamentele descrise în secțiunile de fizică. *Step once* execută un singur pas. *Export motion* scrie state-ul curent în `generated/reports/viewer/motion.csv`.

Camera & impact display. *Orbit*, *Elevation* și *Zoom* mută camera. *AABB*, *OBB* și *Velocity* afișează debug overlays pentru limitele geometrice și vectorii de viteză. *Impact heat coloring* activează heatmap-ul. *Keep impact marks* oprește îmbătrânirea urmelor, *Fade time* stabilește durata lor, *Red at energy* controlează pragul de roșu, iar *Clear impact marks* șterge urmele existente. Legenda arată energia, viteza de apropiere și impulsul normal pentru ultimul impact înregistrat.

Performance, Bodies și Session log. *Performance* arată timpul ultimului pas de simulare, timpul de render și numărul de contacte. *Bodies* este numele panoului în UI. Poți selecta un object după ID și citi poziția și modulul vitezei. *Session log* afișează evenimente precum schimbarea backend-ului, spawn și export, cu un câmp de filtrare.

Controalele și datele afișate sunt definite în **C31** și **C32**.

Fragment C33. Comută vizibilitatea panourilor din meniul Widgets.

```
395  ImGui::MenuItem("Simulation", nullptr, &view.simulationWidget);
396  ImGui::MenuItem("Spawn objects", nullptr, &view.spawnWidget);
397  ImGui::MenuItem("Physics settings", nullptr, &view.physicsWidget);
398  ImGui::MenuItem("Camera & impact display", nullptr, &view.cameraWidget);
399  ImGui::MenuItem("Performance", nullptr, &view.performanceWidget);
```

Sursă C33 `src/client/physics_app/settings/parameter_windows.cpp`
Linii 395–399

Pentru a urmări o singură coliziune, alegi scenariul de impact, pui scena pe pauză și activezi săgețile de viteză și urmele. *Step once* avansează un pas. Săgețile arată schimbarea mișcării, iar urmele arată locul impulsului confirmat.

Fragment C34. Avansează simularea, descarcă starea și actualizează urmele.

```
117  void stepSimulation(Session& session) {
118      session.backend->setImpactTracking(session.impacts().enabled);
119      session.simulation.result = session.backend->step(session.simulation.state,
         session.scene.world);
120
121      session.backend->download(session.simulation.state);
122      session.impacts().advance(session.simulation.state, session.backend->impacts(),
         session.scene.world.dt_s);
123  }
```

Sursă C34 `src/client/physics_app/run_session.cpp`
Linii 117–123

8 Build și rulare

Proiectul folosește CMake 3.27 sau mai nou, Ninja, Git și un compiler cu suport C++20. Pe Windows, scriptul de build detectează MSVC și Windows SDK. Pentru CUDA sunt necesare un GPU NVIDIA, driverul corespunzător și CUDA Toolkit 12.4 sau mai nou, cu un host compiler acceptat de `nvcc`. Viewer-ul rulează cu OpenGL 4.6. Build-ul viewer-ului folosește Python 3

pentru generarea loader-ului GLAD. CMake descarcă dependințele prin *FetchContent*, inclusiv nlohmann/json și, pentru viewer, GLFW, GLM, GLAD și Dear ImGui.

8.1 Build pe Windows

Din directorul principal al proiectului, rulează în PowerShell.

```
.\scripts\deployment\build-windows.ps1 -Cuda -Viewer -Architecture 89
.\exports\builds\windows-x64\physics_viewer.exe --backend cuda --scenario A
```

Scriptul configurează un *Release build* cu Ninja în `tmp/build/windows`. `-Cuda` activează `ENABLE_CUDA`, iar `-Viewer` activează `BUILD_VIEWER`. `-Architecture 89` selectează SM 89, folosit în benchmark. Pentru alt GPU, alege valoarea *compute capability* corespunzătoare acelui model. După build, directorul `exports/builds/windows-x64` conține `physics.exe`, `physics_viewer.exe` și DLL-urile de runtime copiate de script.

8.2 Build pe Linux

Din directorul principal al proiectului, rulează în Bash.

```
bash ./scripts/deployment/build-linux.sh --cuda --viewer --arch 89
./exports/builds/linux-$(uname -m)/physics_viewer --backend cuda --scenario A
```

Scriptul folosește `tmp/build/linux` ca *build directory* implicit și pune build outputs în `exports/builds/linux-` urmat de arhitectura returnată de `uname -m`. Pentru un build CPU-only, omite `-Cuda` pe Windows sau `--cuda` pe Linux și pornește viewer-ul cu `--backend cpu`. Target-ul `physics` este CLI-ul pentru benchmark și replay. Target-ul `physics_viewer` apare când build-ul include viewer-ul. Configurarea și target-urile sunt definite în **C35**, **C36** și **C37**.

9 Validare experimentală

9.1 Metodă

Executabilul Windows x64 a fost rulat în configurația Release la 9 noiembrie 2026 pe un Intel Core i9-14900HX și un NVIDIA GeForce RTX 4070 Laptop GPU. Versiunile au fost MSVC 19.51, CUDA 13.4 pentru SM 89 și driverul 617.42. Starea a fost reprezentată în precizie `float32`, cu *random seed* 42 și pasul fix 1/120 s. Scenariul demonstrativ a rulat 2 400 de pași, iar matricile A, B și C câte 720. Fiecare implementare a rulat independent din aceleași stări inițiale. Figura 4 arată cele trei densități.

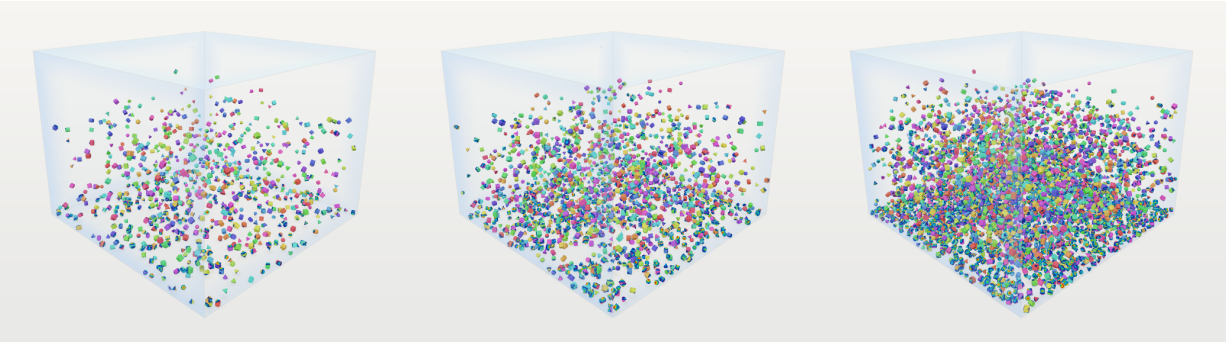


Figura 4: De la stânga la dreapta, matricile A, B și C, cu 850, 1 750 și 4 000 de objects.
Imagine din proiect `media/matrices/cuda-A.png`, `media/matrices/cuda-B.png` și `media/matrices/cuda-C.png`.

Timpii provin din coloana `total_wall_ms` a fișierului `frames.csv`. Măsurarea include primul pas și folosește detectarea urmelor de impact dezactivată. Percentila 95 este interpolată liniar

între timpii ordonați. Timpii includ sincronizarea diagnosticului CUDA. Descărcarea completă a stării și randarea se măsoară separat de pasul fizic.

9.2 Rezultate

Scenariu	Objects	Pași identici	Contacte cumulate	Hash final
Demo	120	2 400/2 400	1 618 414	15956118138859661966
Matrice A	850	720/720	3 153 888	3330260331900169050
Matrice B	1 750	720/720	5 966 806	3349342891293941449
Matrice C	4 000	720/720	12 165 191	11731164910491628103

Tabelul 2: Comparație între CPU și CUDA. Contactele cumulate reprezintă suma numărului de constrângeri pe toți pașii și coincid între implementări.

Hash-urile coincid pentru toți cei 4 560 de pași comparați. La sfârșitul pașilor, ambele implementări au raportat zero ieșiri din incintă. Diagnosticul nu a găsit poziții sau viteze nefinite. Cea mai mare proiecție finală la un perete a fost de aproximativ 0,138 m, în matricea B. Ea măsoară o corecție geometrică din timpul pasului, iar ieșirile se verifică după corecție.

	CPU (ms/pas)		CUDA (ms/pas)		
Scenariu	Mediană	P95	Mediană	P95	**CPU/CUDA**
Demo	0,901	1,088	1,102	1,140	0,82
Matrice A	6,532	8,016	5,238	5,784	1,25
Matrice B	12,807	17,880	9,275	11,190	1,38
Matrice C	31,675	44,259	19,908	25,376	1,59

Tabelul 3: Timpii pasului fizic. P95 este percentila 95, iar ultima coloană este mediana CPU împărțită la mediana CUDA.

În tabelul 3, CUDA este mai rapid la toate cele trei matrici. Scena cu 120 de objects favorizează CPU, deoarece costul lansării și sincronizării CUDA cântărește mai mult când volumul de calcul este mic. La matricea C, timpul pentru *broad phase* scade de la 3,722 ms la 0,616 ms. Rezolvarea contactelor rămâne etapa dominantă, cu mediane de 23,700 ms pe CPU și 16,275 ms pe CUDA. Diagnosticul serial CUDA are mediana 2,781 ms. Medianele etapelor sunt calculate separat. Demo și matricea A se încadrează la mediană în bugetul de 8,33 ms pe pas pe ambele implementări.

Testele unitare acoperă toate cele zece perechi de forme, cei șase pereți, *continuous collision detection* pentru sfere rapide, repausul, reactivarea și reluarea deterministă. Comparația CPU-/CUDA cu cinci valori pentru *random seed* a avut abatere finală nulă pentru poziții și viteze. Testul CPU de un milion de pași pentru o stivă a trecut, iar cutia de sus s-a stabilizat la aproximativ 3,1 mm sub poziția inițială. Testele de impact au comparat și mișcarea cu și fără înregistrarea urmelor.

9.3 Reproducerea comparației

După compilarea executabilului, rulează comenzile PowerShell din directorul principal al proiectului. Înlocuiește `A` cu `B` sau `C` pentru o scenă mai mare.

```
$physics = '.\exports\builds\windows-x64\physics.exe'
$out = 'generated/reports/matrix-a-cpu'
$argsForRun = @('--backend','cpu','--scenario','A','--seed','42',
                '--ticks','720','--output',$out)
& $physics @argsForRun
& $physics --backend cuda --replay $out
```

Rularea scrie `config.json`, `initial.json`, `final.json`, `trace.csv`, `frames.csv`, `motion.csv` și `manifest.json`. Manifestul include *source fingerprint* în câmpul `sourceSha256` și datele de execuție. La configurare, CMake calculează SHA-256 pentru fiecare fișier sursă și de configurare selectat, apoi calculează SHA-256 din concatenarea acestor hash-uri (**C38**). O rulare separată cu `--backend cuda` și un director propriu de ieșire produce timpii CUDA. *Source fingerprint* versiunii testate este

`257b984168a1014c5c034a6dcf075c1b3fd86f31a49f4038d688c9031ae7544e`

10 Concluzii

Rezolvarea contactelor în aceeași ordine permite folosirea implementării CPU ca referință pentru CUDA. În scenele mai mari, căutarea paralelă reduce timpul de detecție, iar rezolvarea constrângerilor dintre objects rămâne costul dominant. Viewer-ul leagă rezultatul fizic de imagine prin orientarea object-urilor, umbrele de contact și urmele de impact, fiecare calculată din datele pasului de simulare.

Sursele proiectului

Căile fragmentelor și ale imaginilor sunt relative la proiectul *Chasing the Moment of Impact* (`chasing-moment-of-impact`). Fragmentele păstrează codul și numerotarea din sursă. Liniile lungi sunt împărțite vizual pentru a încăpea în pagină.

Indexul surselor de cod

Apasă pe un identificator din index pentru a reveni la fragmentul sau la pasajul corespunzător. Identificatorul din text duce la fișierul și intervalul de linii din index.

ID	Fișier sursă	Linii
C1	src/shared/sim/state.cpp	105–106, 137–147
C2	src/shared/sim/state.h	107–112
C3	src/shared/sim/dynamics.h	81–85
C4	src/shared/sim/quaternion.h	81–83
C5	src/shared/sim/diagnostics.h	56–58
C6	src/shared/sim/broad_phase.h	135–138
C7	src/shared/sim/contacts.h	35–40
C8	src/shared/sim/contacts.h	32–71, 87–129
C9	src/shared/sim/contacts.h	180–184
C10	src/shared/sim/manifolds.h	204–212
C11	src/shared/sim/intersections.h	53–75
C12	src/shared/sim/intersections.h	86–133
C13	tests/unit/test_main.cpp	170–183

ID	Fișier sursă	Linii
C14	src/shared/sim/contacts.h	54–60
C15	src/shared/sim/manifolds.h	252–344
C16	src/shared/sim/solver.h	27–30
C17	src/shared/sim/solver.h	138–145
C18	src/shared/sim/solver.h	53–55
C19	src/shared/sim/solver.h	165–172
C20	src/shared/sim/solver.h	192–231
C21	src/physics/backend/cuda/solver.cu	68–75
C22	src/client/render_gl/rigid_bodies/draw_objects.cpp	116–120
C23	src/client/render_gl/rigid_bodies/draw_environment.cpp	95–119
C24	src/client/render_gl/shaders/shadow.frag	41–49
C25	src/physics/scenarios/impact_demo.cpp	22–35, 55–85
C26	src/shared/sim/impacts.h	60–64
C27	src/client/render_gl/impact_history.cpp	80–93, 140–165
C28	src/client/render_gl/shaders/common/impact.glsl	66–70
C29	src/client/render_gl/shaders/common/impact.glsl	41–45
C30	src/client/render_gl/shaders/body.frag	141–145
C31	src/client/physics_app/settings/parameter_windows.cpp	87–130, 154–241, 278–405, 417–507
C32	src/client/physics_app/settings/diagnostic_windows.cpp	34–52, 65–112
C33	src/client/physics_app/settings/parameter_windows.cpp	395–399
C34	src/client/physics_app/run_session.cpp	117–123
C35	scripts/deployment/build-windows.ps1	274–319
C36	scripts/deployment/build-linux.sh	150–183
C37	infra/Executable.cmake	1–4, 30–42
C38	infra/SourceFingerprint.cmake	5–13

Bibliografie

[1] D. Agrawal. Implementing your own 2d game engine. Medium. Testul Circle-AABB.

[2] E. S. Baltacı. Cpu raytracer in c# – part 1. Medium, 2025. Secțiunea Triangle Intersection.

[3] E. Catto. Fast and simple physics using sequential impulses. In *Game Developers Conference*, 2006.

[4] S. Gottschalk, M. C. Lin, and D. Manocha. Obbtree: A hierarchical structure for rapid interference detection. In *Proceedings of SIGGRAPH*, pages 171–180, 1996.

[5] Khronos Group. *The OpenGL Shading Language*, version 4.60 edition. Secțiunea 8.3, smoothstep.

[6] M. Mizerski. Improving simulation and performance with an advanced physics solver. Medium, 2020. Secțiunile Solving și Getting More Details.

[7] T. Möller and B. Trumbore. Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, 2(1):21–28, 1997.

[8] NVIDIA. Cuda core compute libraries, cub deviceradixsort. Documentație API și Cod-sursă.

[9] Rustgodgaming. Building a 2d game engine from scratch: A deep dive into bdengine2d. Medium. Secțiunea The Physics System.